

Лекция 8. Централизованный мониторинг Linux-инфраструктуры: Zabbix

Мониторинг нужен, чтобы обнаружить проблему до жалобы пользователя

Zabbix собирает метрики, вычисляет условия проблем и помогает организовать реакцию

Учебный стенд: Zabbix Server наблюдает web01, vpn01 и backup01

Главная идея: хорошая проверка должна быть измеримой, объяснимой и привязанной к действию администратора

Цель лекции

Понять отличие мониторинга от журналирования, аудита и диагностики

Уметь объяснить роли Zabbix Server, Agent, Proxy, Database и Frontend

Уметь различать Host, Template, Item, Trigger, Problem, Severity и Maintenance

Уметь проектировать проверки CPU, RAM, диска, сети, службы, приложения и backup

Уметь объяснить ограничения UserParameter, уведомлений и безопасности Zabbix

Учебные вопросы

1. Мониторинг, журналирование и аудит
2. Zabbix Server, Agent, Proxy, Database и Frontend
3. Host, Host Group, Template, Item, Trigger и Severity
4. Графики, dashboards, problems, acknowledge и maintenance
5. CPU, RAM, диски, сеть, процессы, порты и службы
6. Active/passive checks, пороги и ложные срабатывания
7. Уведомления, backup, пользовательские сценарии и безопасность Zabbix

Учебный сценарий лекции

web01: проверяем Nginx, HTTP-ответ, CPU, RAM, диск и inode

vpn01: проверяем процесс OpenVPN, UDP 1194 и признаки реального подключения

backup01: проверяем возраст последнего успешного backup, место и состояние repository

Zabbix Server: проверяем очередь, unsupported items, database и собственные процессы

Лаборатория строится через матрицу отказов и ожидаемых проблем

1. Мониторинг, журналирование и аудит

Мониторинг отвечает на вопрос: есть ли сейчас отклонение от нормы

Журналирование отвечает на вопрос: какие события уже произошли

Аудит отвечает на вопрос: кто выполнил действие и было ли оно разрешено

Диагностика объясняет причину после обнаружения симптома

Zabbix в первую очередь решает задачи мониторинга и оповещения

Мониторинг не заменяет диагностику

Trigger может сообщить, что Nginx недоступен

Причину администратор выясняет командами
systemctl, journalctl, ss, curl и tcpdump

Мониторинг должен указывать, где проблема и
насколько она критична

Диагностика должна подтвердить конкретный уровень
отказа

Хороший мониторинг сокращает время поиска, но не
думает за администратора

2. Zabbix Server, Agent, Proxy, Database и Frontend

Zabbix Server собирает значения, обрабатывает items и рассчитывает triggers

Zabbix Agent установлен на наблюдаемом Linux-сервере

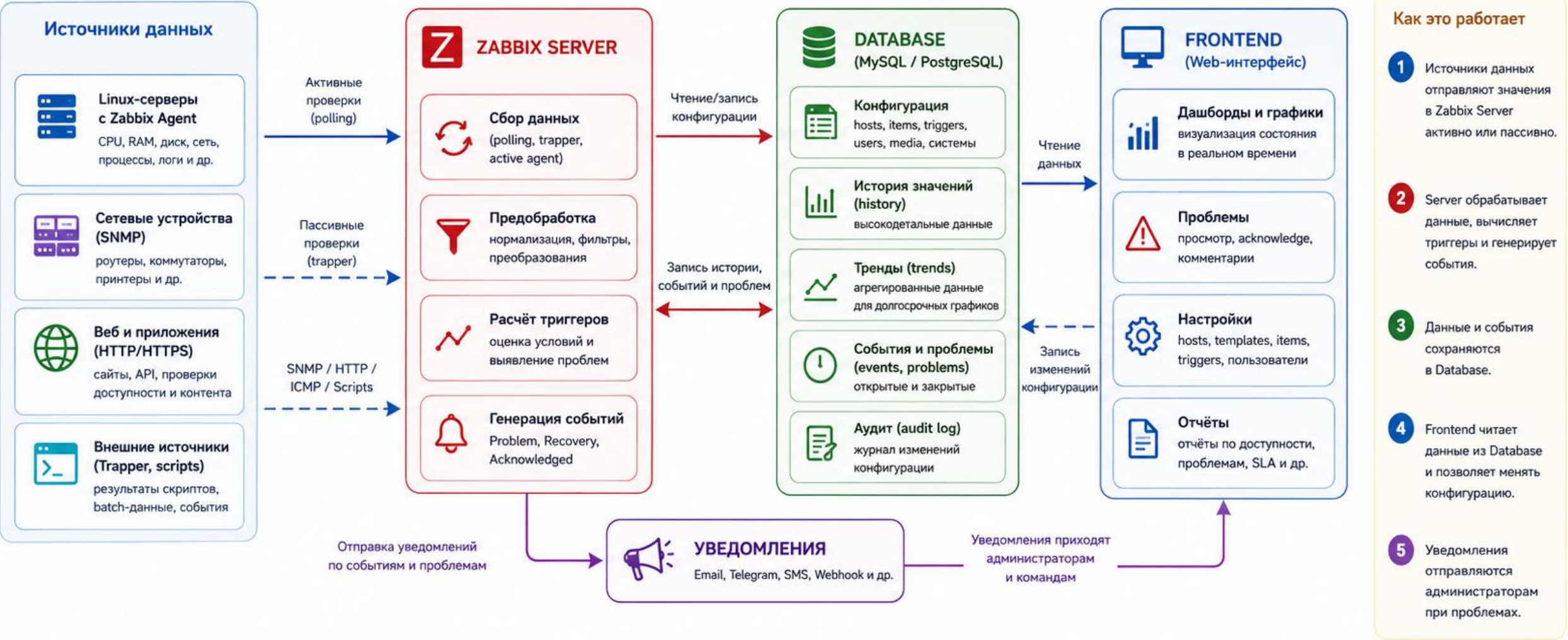
Database хранит конфигурацию, history, trends, events и problems

Frontend показывает web-интерфейс и изменяет конфигурацию через database

Frontend сам не собирает метрики с агентов

Схема: поток данных Zabbix Agent, Server, Database и Frontend

Как метрики попадают от устройств в интерфейс и превращаются в проблемы и уведомления



Ключевые принципы



Единая точка обработки
Вычисление триггеров происходит только на Zabbix Server.



Надёжное хранение
Database хранит историю, тренды, события и конфигурацию.



Frontend не собирает данные
Он только отображает и изменяет конфигурацию через Database.



Безопасность и доступ
Доступ к Frontend защищён, а агенты используют ключи.

Поток данных нужно понимать точно

Agent, SNMP, HTTP и ICMP дают исходные значения
Zabbix Server выполняет polling, trapper-прием,
preprocessing и расчет triggers

Database хранит конфигурацию, историю значений и
события

Frontend читает database и позволяет администратору
менять настройки

Если database недоступна, Zabbix теряет возможность
нормально хранить и показывать состояние

Zabbix Proxu нужен для филиалов, DMZ и масштабирования

Proxu собирает данные ближе к удаленной площадке или изолированной зоне

Proxu передает данные на Zabbix Server

Proxu полезен, когда Server не должен напрямую ходить ко всем агентам

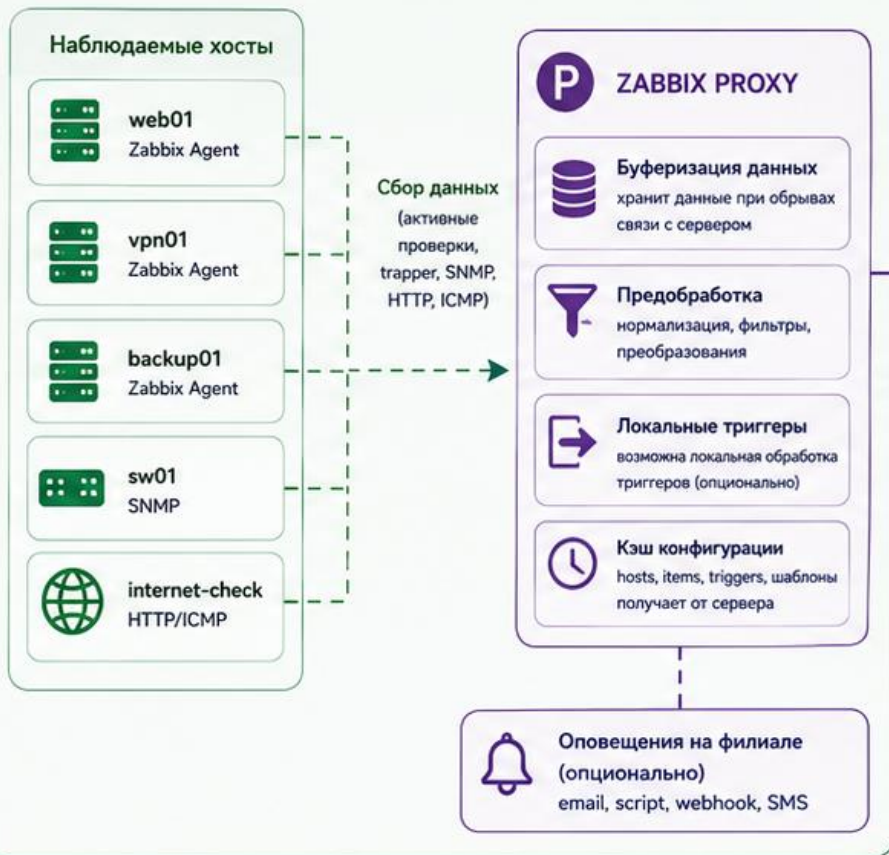
Proxu снижает влияние нестабильного канала между филиалом и центральным сервером

В базовой лаборатории Proxu можно рассмотреть обзорно без установки

Схема: Zabbix Proxy между филиалом и центральным сервером

Proxy собирает данные на стороне филиала и передаёт их на Zabbix Server. Server управляет и хранит всё в базе данных.

ФИЛИАЛ / УДАЛЁННАЯ ПЛОЩАДКА



ЦЕНТРАЛЬНЫЙ СЕРВЕР / ДАТА-ЦЕНТР



Как это работает

1. Агенты в филиале отправляют данные на Zabbix Proxy (локальный сбор).
2. Проxy буферизирует и преобразовывает данные, хранит кэш конфигурации.
3. Проxy передаёт данные на Zabbix Server через исходящее соединение (TCP 10051).
4. Server обрабатывает данные, вычисляет триггеры и сохраняет в Database.
5. Frontend читает данные из Database и предоставляет их администраторам.

Важно!
Проxy не требует входящих соединений из дата-центра. Нужен только исходящий канал к Zabbix Server.

Зачем нужен Proxy



Снижение нагрузки на канал
Передаёт агрегированные данные, а не каждый запрос.



Работа при обрывах связи
Данные сохраняются в буфере и отправляются позже.



Безопасность
Server не обращается напрямую к устройствам в филиале.



Масштабирование
Поддерживает тысячи хостов на удалённых площадках.



Изоляция сети
Подходит для DMZ и сложных сетевых схем.

Agent и Agent 2 нужно выбрать заранее

Zabbix Agent — классический легкий агент

Zabbix Agent 2 — современная реализация с plugin-архитектурой

Набор keys и integration plugins может отличаться

Пакеты и unit names различаются: zabbix-agent и zabbix-agent2

В лаборатории фиксируем один вариант, чтобы не смешивать пути конфигурации и команды

Базовая конфигурация Agent 2

Файл: `/etc/zabbix/zabbix_agent2.conf`

Параметр `Server=192.168.30.40` разрешает passive-запросы от Zabbix Server

Параметр `ServerActive=192.168.30.40` задает сервер для active checks

Параметр `Hostname=web01` должен совпадать с Host name во Frontend для active checks

После изменения: `sudo systemctl restart zabbix-agent2`

Passive check инициирует Server или Proxy

Passive check: Server или Proxy подключается к Agent

Типичный порт: Agent TCP 10050

Пример проверки с сервера: `zabbix_get -s 192.168.30.10 -k system.hostname`

Плюс: удобно в управляемой внутренней сети

Минус: нужен входящий доступ к Agent

Active check инициирует Agent

Agent подключается к Server или Proxy на TCP 10051

Agent запрашивает список active items

Agent сам выполняет проверки

Agent отправляет собранные значения обратно на Server или Proxy

Active checks удобны при NAT, DMZ, филиалах и закрытом входящем доступе

Active check не отменяет все сетевые требования

Если host использует только active items, TCP 10050 может не требоваться

При смешанном template могут остаться passive agent items

Remote commands и некоторые проверки могут требовать подключения к Agent

Сетевые требования определяются каждым типом item

Для active checks критично совпадение Agent Hostname и Host name во Frontend

Hostname важен именно для active checks

В Agent: Hostname=web01

В Agent: ServerActive=192.168.30.40

Во Frontend: Host name должен быть web01

Visible name может отличаться и служит для удобного отображения

Типичная ошибка: Agent Hostname web01, а Host name во Frontend WEB01

3. Host, Host Group, Template, Item, Trigger и Severity

Host — наблюдаемый объект: сервер, устройство, сервис или виртуальная машина

Host Group помогает группировать объекты и назначать права

Template хранит типовые items, triggers, graphs и discovery rules

Item собирает одно конкретное значение

Trigger превращает значение item в состояние Problem

Item должен возвращать одно значение

Item имеет key, type, interval, type of information и history period

Пример key: system.cpu.load[all,avg1]

Пример key: vfs.fs.size[/,pused]

Пример key: net.if.in[ens33]

Один item не должен возвращать длинный
человекочитаемый отчет

Item может быть Supported, Unsupported или No Data

Supported означает, что item получает корректные значения

Unsupported означает, что Zabbix не смог выполнить item или разобрать результат

No Data означает отсутствие новых значений за ожидаемый период

Отсутствие данных само должно контролироваться отдельным trigger

Пример: `nodata(/web01/agent.ping,5m)=1`

Low-Level Discovery создает items по найденным объектам

Discovery rule может найти файловые системы, сетевые интерфейсы или приложения

Item Prototype описывает, какие items создавать для каждого найденного объекта

Trigger Prototype описывает, какие triggers создавать автоматически

Это удобно для дисков и сетевых интерфейсов, которые отличаются на разных серверах

В базовой лекции достаточно понимать идею без глубокой настройки

Trigger должен учитывать длительность, а не только последнее число

last() проверяет последнее значение

min() проверяет минимальное значение за интервал

max() проверяет максимальное значение за интервал

avg() проверяет среднее значение за интервал

nodata() проверяет отсутствие данных

timeleft() помогает прогнозировать достижение порога

Пример trigger с длительностью

Плохая идея: открыть Problem из-за одного краткого скачка CPU

Лучше: средняя загрузка выше порога в течение 10 минут

Концептуальный пример:

`avg(/web01/system.cpu.load[all,avg1],10m)>4`

Для диска: `min(/web01/vfs.fs.size[/,pfree],5m)<10`

Точный синтаксис нужно сверять с версией Zabbix в лаборатории

Hysteresis снижает дрожание Problem и Recovery

Если Problem открывается при 90%, закрытие при 89.9% создаст постоянные переключения

Лучше открыть Problem при $\text{usage} > 90\%$

Закрыть Problem при $\text{usage} < 85\%$

В Zabbix это реализуют recovery expression или устойчивой trigger logic

Hysteresis особенно важен для диска, CPU и сетевой задержки

Severity отражает влияние, а не тип метрики

Nginx остановлен на test-web01 может быть Warning

Nginx остановлен на production-web01 может быть High

Диск 90% на временном сервере и на базе данных имеют разный риск

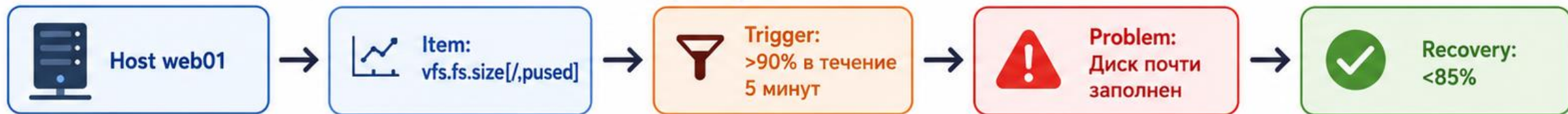
Severity зависит от роли host, SLA, времени суток и наличия резервирования

Порог должен вести к правильному приоритету реакции

Диаграмма: Host, Item, Trigger, Problem и Recovery



Пример логики



Ключевая идея



Host содержит items



Trigger превращает значение item в состояние Problem



Recovery закрывает проблему после нормализации

4. Graphs, dashboards, problems, acknowledge и maintenance

Problems показывают текущие события, требующие внимания

Graphs помогают понять динамику значения во времени

Dashboards собирают эксплуатационную картину на одном экране

Acknowledge означает, что администратор увидел проблему

Maintenance используется для согласованных работ, а не для скрытия неудобных тревог

Problem, Acknowledge, Recovery и Manual Close различаются

Problem открыт, когда выполняется trigger condition

Acknowledge фиксирует, что администратор взял проблему в работу

Recovery происходит, когда recovery condition перестала быть проблемной

Manual close возможен только для специально настроенных triggers

Maintenance может подавлять уведомления во время плановых работ

Tags помогают фильтровать и маршрутизировать проблемы

Tag может указывать сервис: `service=nginx` или `service=backup`

Tag может указывать среду: `env=lab`, `env=prod`

Tag может указывать команду реакции: `team=linux`

Actions могут использовать tags для выбора получателей

Без tags список проблем быстро превращается в неуправляемую ленту

Dashboard должен отвечать на эксплуатационные вопросы

Какие hosts недоступны?

Какие active problems сейчас самые критичные?

Где растёт disk usage или inode usage?

Когда был последний успешный backup?

Есть ли очередь Zabbix и unsupported items?

5. CPU, RAM, диски, сеть, процессы, порты и службы

Базовые Linux-метрики должны показывать не только факт нагрузки, но и тип риска

CPU, RAM, disk и network требуют разных интерпретаций

Process, unit, port и application check нужно проверять разными уровнями

Для каждого сервиса важно отделять локальное состояние от внешней доступности

Мониторинг должен помогать выбрать следующую диагностическую команду

CPU нужно читать через несколько признаков

CPU utilization показывает занятость процессора

Load average показывает очередь работы и зависит от числа CPU

I/O wait показывает ожидание дисковой подсистемы

Steal time важен для виртуальных машин и показывает дефицит CPU на гипервизоре

Высокий load не всегда означает проблему именно с процессором

RAM нельзя оценивать только по used percent

Linux активно использует память под page cache

Available memory полезнее для тревоги, чем просто used memory

Swap usage и swap in/out показывают давление памяти

OOM events показывают, что ядро убивало процессы из-за нехватки RAM

Критическая тревога только по высокому cache может быть ложной

Диск требует проверки capacity, inode и I/O

Filesystem usage показывает заполнение по байтам

Inode usage показывает риск No space left on device при наличии свободных байтов

I/O latency и utilization показывают проблему производительности

Read-only remount может означать серьезную ошибку файловой системы

RAID degraded требует отдельного контроля

Process, Unit, Port и Application Check нужно разделять

Process exists: процесс присутствует в системе

systemd unit active: служба считает себя запущенной

Local port listening: порт слушается на сервере

Remote port reachable: порт доступен с точки проверки

Application response: сервис возвращает правильный
ответ

Ngинх проверяют минимум по трем уровням

systemd: nginx unit active или inactive

Port: TCP 80 слушается локально

Remote reachability: TCP 80 доступен с Zabbix Server
или Proху

Application: HTTP возвращает 200 и ожидаемое
содержимое

Upstream: приложение за Nginх отвечает без HTTP 500

Локальный port check не доказывает внешнюю доступность

Agent может видеть, что TCP 80 слушается локально

Firewall может блокировать доступ извне

NAT, ACL, маршрут или load balancer могут быть настроены неверно

DNS может указывать не туда

Для критичного сервиса нужны локальная проверка и внешняя web scenario или remote port check

UDP OpenVPN нельзя проверять как обычный TCP-порт

UDP не устанавливает соединение как TCP

Отсутствие ответа по UDP не всегда означает закрытый порт

Нужно проверять процесс OpenVPN и локальное прослушивание UDP 1194

Нужно проверять входящие пакеты, журналы подключения и состояние tunnel

Лучший прикладной тест: тестовый клиент устанавливает VPN и видит внутренний ресурс

6. Active/passive checks, пороги и ложные срабатывания

Тип проверки выбирают по сетевой схеме и требованиям доступа

Порог должен учитывать baseline, длительность и влияние на сервис

Ложные срабатывания снижают доверие к мониторингу

Dependencies помогают подавить вторичные проблемы

Maintenance нужен для плановых работ, а не для маскировки ошибок

Dependencies уменьшают alarm storm

Если router недоступен, web01, vrn01 и backup01 тоже могут стать недоступны

Без dependencies Zabbix создаст много вторичных problems

Основная проблема: Host unreachable через router

Зависимые проблемы: services on hosts

Зависимости помогают дежурному видеть корневой симптом первым

Уведомления, backup, пользовательские сценарии и безопасность Zabbix

Уведомление должно содержать данные для действия, а не только текст Alarm

Backup нужно мониторить по подтвержденной успешной точке, а не просто по свежему файлу
UserParameter должен возвращать одно машинно читаемое значение

Скрипты Agent выполняются с ограниченными правами

Сам Zabbix Server является критичной системой и тоже должен мониториться

Backup age должен измерять успешную копию

Свежий файл не доказывает успешный backup

Файл может быть пустым, поврежденным или созданным до завершения задания

Backup script должен создавать marker только после успешного завершения

Полезные items: backup.last_success_timestamp, backup.duration, backup.size

Дополнительно контролируют files_count, integrity_status и restore_test_age

UserParameter получает stdout, а не exit code как значение

Обычный UserParameter передает в Zabbix стандартный вывод команды

Exit code нужен Agent для понимания успешности выполнения команды

Exit code не превращается автоматически в item value 0, 1, 2 или 3

Если item ожидает число, скрипт должен вывести только число

Человекочитаемый вывод вида CRITICAL: disk=95% может сделать item unsupported

Скрипт для Numeric Item должен печатать число

Пример item: custom.backup.age

UserParameter=custom.backup.age,/usr/local/sbin/backup-age-zabbix.sh

Скрипт выводит возраст последнего успешного backup в секундах

Если marker отсутствует, скрипт может вывести большое число, например 999999

Trigger сравнивает item с RPO, например backup age > 86400

server-health.sh и Zabbix Items лучше разделить

server-health.sh удобен человеку как общий отчет

Zabbix item должен возвращать одно значение

Примеры items: custom.disk.usage[/],

custom.service.status[nginx]

Примеры items: custom.backup.age, custom.raid.state

Разделение упрощает trigger expressions и снижает
риск unsupported items

Гибкий UserParameter требует валидации аргументов

Пример:

```
UserParameter=custom.service.status[*],/usr/local/sbin/c  
heck-service-zabbix.sh "$1"
```

Аргумент \$1 поступает из item key

Скрипт должен разрешать только ожидаемые unit
names

Разрешенный список: nginx, ssh, openvpn-
server@server

Для неизвестного значения вернуть

ZBX_NOTSUPPORTED: unsupported service

UserParameter выполняется с правами пользователя zabbix

Скрипт не выполняется от root по умолчанию

Пользователь zabbix может не читать закрытые журналы, /root, docker socket и private keys

Нельзя выдавать zabbix ALL=(ALL) NOPASSWD: ALL

Если нужны повышенные права, используют точечный root-owned wrapper

Wrapper должен возвращать только нужное значение и не принимать опасные аргументы

UserParameter нужно тестировать от имени Agent

Недостаточно запустить скрипт от root

Команда: `sudo -u zabbix /usr/local/sbin/backup-agent-zabbix.sh`

Назначение: проверить реальные права пользователя Agent

После изменения конфигурации: `sudo systemctl restart zabbix-agent2`

Для classic agent команда будет отличаться: `sudo systemctl restart zabbix-agent`

Тяжелые проверки нельзя выполнять синхронно из Agent

UserParameter не должен запускать длительный `du /` или полный `find`

UserParameter не должен запускать `backup` или `restore`

Долгая команда может превысить `timeout` и сделать `item unsupported`

Лучше: `systemd timer` выполняет тяжелую проверку и пишет `state file`

UserParameter быстро читает готовое число из `state file`

Action, Media Type, Escalation и Recovery Notification различаются

Media Type — способ доставки: email, webhook или другой канал

User Media — адрес конкретного пользователя

Action — условия и операции отправки уведомлений

Escalation — повторные или изменяющиеся действия со временем

Recovery Notification сообщает, что проблема действительно завершилась

В сообщении нужны host, severity, last value, duration, tags и ссылка на runbook

Безопасность Agent-соединений требует ограничений

Server= и ServerActive= должны указывать доверенные адреса

TCP 10050 открывают только с Zabbix Server или Proxy
Active checks подключаются к TCP 10051 на Server или Proxy

Через недоверенные сети используют TLS PSK или certificates

Agent не выставляют напрямую в Интернет и не включают remote commands без необходимости

Zabbix Frontend — отдельная поверхность атаки

В маленьком стенде Server, Database и Frontend могут быть на одной VM

В production их рассматривают как разные зоны доверия

Frontend должен работать через HTTPS

Нужны сильная аутентификация, RBAC и отдельные admin accounts

Нужно защищать database credentials и журналировать входы

Секреты в Zabbix нужно хранить как Secret Macros

В Zabbix могут храниться tokens, SNMP communities, API keys и passwords

Обычные user macros могут быть видны шире, чем нужно

Secret macros скрывают значение в интерфейсе

Секреты нельзя выводить в trigger name, logs и operational data

Credentials должны иметь процедуру ротации

Backup Zabbix включает больше, чем database

Основная конфигурация Zabbix хранится в database

Также нужны server config, frontend config и web server config

Нужны TLS keys, PSK, custom scripts и alert scripts

Нужны UserParameter configurations и external checks

Template export полезен, но не заменяет backup database

Сам Zabbix должен мониториться

Проверять доступность Zabbix Server process

Проверять queue, poller busy, preprocessing busy и cache usage

Проверять database connections, database size и disk space

Проверять unsupported items и housekeeper duration

Для proxy проверять proxy last seen

Смоделировать заполнение диска нужно безопасно

Не заполнять root filesystem рабочей VM до 95–100%

Создать отдельный небольшой LV или виртуальный диск

Смонтировать его в /srv/monitor-test

Создать trigger только для этой файловой системы

После Problem удалить тестовый файл и проверить Recovery

Матрица отказов делает лабораторию проверяемой

Agent остановлен — expected problem: Agent unavailable

Nginx остановлен — expected problem: Service down

HTTP возвращает 500 — expected problem: Application unavailable

Test filesystem >90% — expected problem: Disk high usage

Backup старше RPO — expected problem: Backup stale

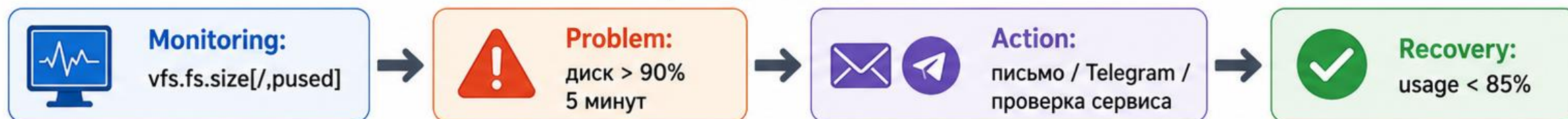
Maintenance active — expected behavior: notification suppressed

Диаграмма: цикл monitoring, problem, action и recovery

Как мониторинг превращает отклонение в действие администратора и возвращается к нормальному состоянию



Пример на практике



Ключевая идея

Monitoring выявляет отклонение, Problem фиксирует его, Action запускает реакцию, Recovery закрывает событие и цикл продолжается.

Итоги лекции

Zabbix полезен, когда метрика связана с порогом, влиянием и действием администратора

Active и passive checks отличаются инициатором соединения, портами и сетевыми требованиями

Trigger должен учитывать длительность, отсутствие данных, hysteresis и влияние на сервис

UserParameter должен возвращать одно машинно читаемое значение и выполняться с ограниченными правами

Мониторить нужно не только Linux hosts, но и backup, Zabbix Server, database и саму систему уведомлений

Контрольные вопросы

Чем мониторинг отличается от журналирования и аудита?

Какой поток данных проходит через Agent, Server, Database и Frontend?

Чем passive check отличается от active check?

Почему локальный port check не доказывает внешнюю доступность сервиса?

Почему exit code UserParameter не является item value?

Какие метрики нужно контролировать для backup и самого Zabbix Server?